

1. 03 — Entorno RL, algoritmo y entrenamiento

1.1. 1) Entorno RL: qué modela exactamente

Archivo: `src/rl_defender_env.py`

- observación: $X[i]$
- acción: 0=PERMIT, 1=BLOCK
- recompensa según etiqueta real y acción

Es una formulación de decisión secuencial simple sobre muestras etiquetadas (muy cercana a contextual bandit con coste asimétrico).

1.2. 2) Recompensa actual por defecto

En código mantenido (`train_rl_defender.py`, `validate_checks.py`, `validate_leave_one_csv_o`

```
{
  "tp": 1.5,
  "fp": -1.5,
  "fn": -5.0,
  "omission": 0.0,
}
```

Interpretación:

- bloquear ataque recompensa moderadamente
- bloquear benigno penaliza
- permitir ataque penaliza fuertemente
- permitir benigno queda neutro (omission)

1.3. 3) Sobre el algoritmo principal

Archivo: `src/train_rl_defender.py`

- se usa QRDQN (`sb3_contrib`)
- política: `MlpPolicy`
- arquitectura de red en entrenamiento principal: [512, 256]
- entorno envuelto con `DummyVecEnv` y `Monitor`

1.4. 4) Hiperparámetros relevantes en entrenamiento principal

- `learning_rate = 1e-4`
- `gamma = 0.99`
- `tau = 1.0`
- `batch_size`: depende de preset (512 fast, 2048 full)
- `gradient_steps`: depende de preset (10 fast, 20 full)

1.5. 5) Presets y timesteps

Código efectivo:

- **fast**: 25_000 timesteps por defecto
- **full**: 100_000 timesteps por defecto

Nota: el texto de ayuda de CLI sobre timesteps puede quedar desactualizado frente al valor real aplicado en `main()`.

1.6. 6) Artefactos que deja cada entrenamiento

- modelo: `models/<RUN_ID>.zip`
- en `runs/cicids2017/<RUN_ID>/`:
 - `config.json`
 - `metrics.json`
 - `scaler.joblib`
 - `train_percentiles.npz`

Esto permite reproducibilidad y paso consistente a Phase 2.

1.7. 7) Baseline y tuning (complemento)

1.7.1. Baseline Random Forest

Archivo: `src/baseline_random_forest.py`

Sirve para comparar contra un modelo supervisado clásico.

1.7.2. Tuning con Optuna

Archivo: `src/tune_hparams.py`

Explora hiperparámetros de QRDQN y optimiza F1 de ataque.